Planning Fast and Slow: Trajectory Synthesis via Conditional Flow Matching

Frank Tsu-Fang Lu

Master of Science in Computer Vision, Robotics and Machine Learning

from the

University of Surrey

School of Computer Science and Electrical and Electronic Engineering

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

September 2025

Supervised by: Dr. Simon Hadfield

DECLARATION OF ORIGINALITY

I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

MSc Dissertation Title: Guided Trajectory Synthesis via Flow Matching

Author Name: Frank Tsu-Fang Lu

Author Signature:  Date: September 2, 2025

Supervisor's name: Simon Hadfield

WORD COUNT

Number of Pages:

Number of Words:

ABSTRACT

Generative modeling has emerged as a powerful paradigm for offline reinforcement learning (RL), with diffusion models showing particular promise for synthesizing complex, multimodal trajectories. However, a fundamental tension exists between their iterative, stochastic nature and the stringent low-latency and deterministic requirements of real-world robotic control. This conflict presents a significant barrier to deploying such models in time-critical, physical systems.

This dissertation introduces Flow Planner, a novel hierarchical framework built upon Conditional Flow Matching (CFM)—a deterministic and computationally efficient alternative to diffusion models. The framework adapts CFM for state–action sequence generation by decomposing the planning problem into two complementary modules: a long-horizon Waypoint Planner for strategic reasoning and a short-horizon Action Planner for reactive control. These planners are seamlessly integrated within a receding-horizon model predictive control (MPC) loop, enabling a unique blend of deliberative and reactive behavior.

Extensive experiments on the Gymnasium LunarLander benchmark demonstrate that Flow Planner achieves substantially faster inference speeds while maintaining competitive, and often more stable, control performance compared to leading diffusion-based and monolithic baselines. In addition to these performance gains, this work contributes a reproducible data curation pipeline and a benchmark dataset with a trained expert agent to support future research. These results establish flow matching as a robust foundation for efficient, modular, and scalable generative planning. The Flow Planner framework offers a principled approach to bridging the gap between offline RL theory and its practical application in real-time robotic systems.

CONTENTS

LIST OF FIGURES

1 INTRODUCTION

Imbuing intelligent agents with the capability for autonomous-decision-making in real-world environments is challenging. For decades, reinforcement learning has offered a robust mathematical framework for acquiring optimal behavioral skills through interactions within an environment to maximize cumulative rewards. The integration of deep neural networks as high-capacity function approximators has propelled RL even farther, leading to remarkable achievements across diverse domains, from mastering complex games to enabling basic robotic locomotion and manipulation. However, the widespread success of modern machine learning, as seen in areas such as computer vision and natural language processing, fundamentally hinges on the availability of large and highly diverse datasets. As seen in Figure 1.1, in conventional, online RL, this data is collected through a continuous interaction loop: the agent executes its current policy in the environment, collects the resulting experiences into a buffer, and then uses this fresh data to update its policy. This cycle of exploration and optimization is repeated until the policy converges. With this paradigm, the acquisition of such extensive data becomes prohibitively impractical for real-world applications. For example, autonomous driving would need to practice in numerous cities for every model update, which is costly, time-consuming, let alone being unsafe. This inherent limitation created a gulf of generalization between lab-bound RL successes and the complex, open-world settings where intelligent decision-making is most critically needed.

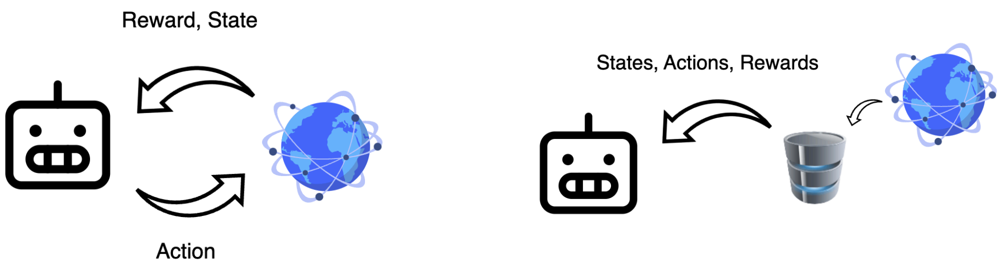

Figure 1.1: Comparison of Online and Offline Reinforcement Learning.

Offline Reinforcement Learning emerges as a pivotal paradigm to bridge this gap. Offline RL focuses on enabling the training of policies solely from previously collected datasets, without additional online interaction. This approach holds immense potential for transforming vast amounts of historical data into robust, generalizable decision-making engines, much like how supervised learning revolutionized pattern recognition. The implications are profound across various fields from robotics, healthcare, and autonomous driving to logistics, finance, and operations research, as it offers a viable pathway to deploy sophisticated AI systems in domains where active data collection is unfeasible, costly, or unsafe.

Finally, the ability to leverage large and diverse datasets in an offline setting aligns the field of robotics learning with the broader paradigm of foundation models, as seen in computer vision and natural language processing. This opens the door to pre-training generalist agents on massive, heterogeneous datasets to acquire a broad repertoire of skills, which can then be efficiently fine-tuned for specific downstream tasks. This paradigm thus offers a compelling vision for the future: transforming vast archives of passive, observational data into active, embodied intelligence.

1.1 Background and Context

Despite its immense promise, offline RL is plagued by algorithmic challenges. At its core, offline RL is fundamentally about making and answering what Levine et al. describe as **"counterfactual queries"** [1]. These are "what if" questions about what might happen if the agent were to carry out actions different from those in the dataset. In other words, for an agent to improve upon the underlying behavior policy in the offline dataset, it must be able to execute action sequences that diverge from the dataset. This inherent need for counterfactual inference strains conventional machine learning tools, which assume that test data is drawn from the same distribution as training data, leading to a problematic distribution shift.

This issue is particularly acute for value-based methods. These methods learn a function to estimate the expected reward, or "goodness", of an action. When queried about an action that is far from the training data, this value estimate has no reliable data to ground its prediction. Function approximation errors can then cause the model to become unreliably optimistic, assigning erroneously high values to these unsupported actions [1]. The policy, trusting these flawed estimates, is then misguided into choosing brittle and unsafe behaviors when deployed.

This instability is particularly exacerbated by high-dimensional function approximators such as Deep Neural Networks (DNNs). The errors can accumulate over long horizons, making stable learning notoriously difficult. Furthermore, real-world datasets, especially those from human demonstrations, inherently exhibit multimodal action distributions: there are often multiple valid ways to perform a task. Traditional methods like behavioral cloning struggle to capture this multimodality, often averaging across modes and leading to indecisive or jerky behaviors that fail to generalize [2].

To address the challenge of multimodality, generative modeling, particularly with denoising diffusion models, has emerged as a powerful approach [3]. By framing planning as a sampling process, diffusion models can learn and represent complex, multi-modal distributions directly from data. This is a critical capability for robotics, where a task may have multiple valid solutions, such as different grasps for an object or various paths around an obstacle. By capturing the entire distribution of expert solutions, these models can enhance a robot's versatility and ability to generalize [2, 4].

However, the very properties that make diffusion models powerful for generation introduce fundamental challenges for their deployment in real-time robotic systems. These models are built upon a stochastic, iterative framework that imposes a significant computational burden, leading to high-latency inference incompatible with dynamic control loops. More importantly, their stochastic nature is fundamentally misaligned with the deterministic physical laws and hard safety constraints that govern robotic motion, calling for an alternative generative paradigm.

Therefore, this dissertation presents Flow Planner, a hierarchical offline RL framework built upon flow matching, a powerful alternative that directly addresses the core limitations of diffusion models [5]. Flow matching learns a deterministic transformation governed by an Ordinary Differential Equation (ODE) rather than a Stochastic Differential Equation (SDE) [6]. This fundamental distinction makes it not merely an incremental improvement but a paradigm inherently aligned with the deterministic, low-latency requirements of physical robotic systems.

1.2 Objectives

The primary aim of this dissertation is to develop and validate a novel, modular framework for imitation learning in offline reinforcement learning, leveraging the principles of flow matching for a high-speed alternative to contemporary diffusion-based planners. The specific objectives established to achieve this aim are as follows:

1. **To develop a specialized Conditional Flow Matching (CFM) formulation for trajectory planning** that efficiently generates coherent state-action sequences for robotic control, adapting techniques previously used in image and video generation.
2. **To architect a hierarchical control framework** using flow-based planning that separates strategic decision-making from tactical execution, effectively solving the scalability and horizon limitations of end-to-end approaches.
3. **To empirically validate the framework** through rigorous benchmarking against monolithic planners and state-of-the-art diffusion models, focusing on control effectiveness, inference speed, and trajectory quality.
4. **To design and implement a reproducible data curation pipeline** for the training and validation of the proposed generative planners, ensuring the work is transparent and reproducible by future research.

1.3 Achievements

The primary aim of this dissertation is to develop and validate a novel, modular framework for imitation learning in offline reinforcement learning, leveraging the principles of flow matching for a high-speed alternative to contemporary diffusion-based planners. The specific objectives established to achieve this aim are as follows:

1. **A Novel Formulation of Flow-Based Trajectory Planning**: This work introduces the first specialization of the Conditional Flow Matching (CFM) framework for the domain of robotic trajectory planning. By reformulating the CFM objective to generate different sequences of states and actions, this contribution establishes a new class of deterministic, high-speed generative planners that directly address the latency limitations of contemporary diffusion-based methods.

2. **A Hierarchical Framework for Long-Horizon Control**: The second key contribution is the Flow Planner architecture, a new two-stage hierarchical framework that decouples long-horizon strategic reasoning from low-level reactive control. This design solves the scaling and coherence problems inherent in monolithic, end-to-end planners by factorizing the planning problem into a strategic Waypoint Planner and a reactive Action Planner, a novel application for flow-based models.

These core contributions are supported by:

1. **A Comprehensive Analysis of Flow-Based Planners**: This work provides an in-depth empirical analysis of applying flow-matching models to robotics, offering insights into the role of different model architectures and conditioning mechanisms, alongside a direct comparison to the diffusion paradigm.

2. **A Reproducible Data Curation Pipeline and Benchmark Datasets**: This work contributes a flexible and extensible pipeline for curating high-quality offline reinforcement learning datasets. This pipeline was used to generate three novel benchmark datasets, which have been publicly released alongside their corresponding trained expert agents to facilitate reproducibility and future research.

These contributions culminate in a robust and efficient framework for imitation learning, with the findings currently being prepared for submission to the IEEE International Conference on Intelligent Robots and Systems (IROS).

1.4 Overview of Dissertation

The remainder of this dissertation is structured as follows:

- **Chapter 2** provides the theoretical foundation for this work, first examining established methods for addressing distribution shift problems, then exploring contemporary solutions that leverage generative models.

- **Chapter 3** presents the core technical foundation of this dissertation. It details various approaches to dataset modeling with comprehensive mathematical formulations.

The chapter ends by introducing ”Flow Planner,” the proposed hierarchical planning framework, and provides implementation specifics using Meta’s Flow Matching library [7].

- **Chapter 4** details the methodology for data curation and introduces the experimental environments. This includes a technical discussion on the implementation of the data pipeline using Gymnasium[8] wrappers and the Minari[9] library for dataset management.
- **Chapter 5** provides a comprehensive analysis and discussion of the empirical results. This includes an investigation of various conditioning mechanisms and a benchmark comparison of the framework’s performance against baseline models.
- **Chapter 6** concludes the dissertation by summarizing the key contributions, discussing the implications of the findings, and outlining several promising directions for future research.

2 BACKGROUND THEORY AND LITERATURE REVIEW

This chapter begins by examining the fundamental concepts in offline reinforcement learning and established methodologies for addressing distributional shift. Next, it explores diffusion models through the lens of variational inference before unifying these concepts through continuous flows to introduce flow matching. The chapter concludes with an analysis of diffusion applications in trajectory synthesis and state-of-the-art approaches that integrate these models directly into reinforcement learning frameworks. Particular emphasis is placed on the mathematical formulations of diffusion and flow matching, as this constitutes the central focus of this dissertation and the applied methodologies after.

2.1 Offline Reinforcement Learning

As established in Chapter 1, a significant hurdle in offline reinforcement learning is the distributional shift between the static dataset and the policy being learned. This can lead to erroneous value estimates for out-of-distribution actions, resulting in brittle policies that perform poorly when deployed [1]. To mitigate this issue, the literature has broadly converged on three primary classes of approach: policy constraint methods, implicit policy constraint methods, and model-based methods.

2.1.1 Policy Constraint Methods

One early class of methods that's been explored extensively is policy constraint methods. These approaches aim to mitigate distributional shift by updating the policy not just by maximizing the Q-value, but by doing so subject to a constraint that the learned policy remains sufficiently close to the behavioral policy (collected data). This closeness is typically measured by a divergence metric, such as KL divergence or Maximum Mean Discrepancy (MMD).

To a degree, these methods can bound the distributional shift and the impact of out-of-distribution actions, potentially leading to more stable learning. Examples include BEAR (Bootstrapping Error Accumulation Reduction) by Kumar et al. [10] and BRAC (Behavior Regularized Actor Critic) by Wu et al. [11].

A significant challenge is their potential to be overly conservative, as a single constraint value ϵ might not work well across all states, especially if the behavior policy exhibits both highly random and highly deterministic actions. More critically, these methods often require accurately estimating the behavior policy itself, which can be extremely difficult, particularly when the data comes from diverse sources (e.g., humans or a mixture of different policies). Errors in this estimation can lead to an imperfect constraint and poor practical performance, making it a major bottleneck, especially during online fine-tuning.

2.1.2 Implicit Policy Constraint Methods

Building upon the insights into the limitations of explicit policy constraint methods, implicit policy constraint methods, such as AWAC (Advantage Weighted Actor Critic) by Nair et al. [12], emerged as an alternative that addresses the difficulty of behavior policy modeling. AWAC leverages a theoretical identity that shows the solution to a constrained optimization problem can be approximated using a weighted maximum likelihood objective. In this formulation, actions taken from the behavior policy are weighted by the exponentiated advantage function, effectively embedding the constraint implicitly [10]. More intuitively, AWAC approximates the best new policy by look at actions already in the dataset and give more weight (advantage) to the really good ones.

This approach obviates the need for explicit modeling of the behavior policy, only requiring samples from it, which are readily available in the offline dataset. AWAC has shown to be effective for online fine-tuning after offline initialization and demonstrates strong performance on complex tasks like dexterous manipulation with human demonstration data.

While the sources highlight its advantages, it is still a form of policy constraint. Its inherent conservativeness might limit the extent to which it can discover entirely novel, optimal behaviors far from the observed data.

2.1.3 Model-based Offline RL Methods

Taking a different algorithmic direction, model-based offline RL methods, exemplified by MOPO (Model-based Offline Policy Optimization) by Yu et al. [13], propose to tackle the problem by learning a model of the environment's dynamics. While vanilla model-based RL (like MBPO [14]) already showed some promise in offline settings, MOPO specifically modifies these algorithms to handle distributional shift. MOPO achieves this by punishing the policy for exploiting out-of-distribution states during model rollouts. This is implemented by subtracting an uncertainty penalty from the reward function, which becomes larger for state-action tuples that are more out of distribution (i.e., where the learned model's predictions are less certain).

MOPO hence has the ability to generalize beyond the state and action support of the offline data, enabling the learning of policies for tasks different from those for which the data was collected. It is more resilient to overestimation and overfitting than model-free methods and theoretically maximizes a lower bound of the policy's true return, balancing exploration and risk. Empirically, MOPO outperforms existing model-free offline RL methods and vanilla model-based approaches, especially on tasks requiring out-of-distribution generalization.

Nevertheless, models can still be inaccurate in regions far from the training data, and in a purely offline setting, these errors cannot be corrected through further interaction. The effectiveness relies heavily on the quality of the uncertainty estimation and the tuning of the penalty coefficient [1]. It may also struggle with datasets lacking sufficient action diversity, and there are limits to its generalization capabilities if the new task requires exploring states completely outside the available data support.

2.1.4 Conservative Q Learning

Finally, a distinct and highly effective approach is Conservative Q-Learning (CQL) by Kumar et al. [15], which directly addresses the core problem of Q-value overestimation. Instead of constraining the policy, CQL augments the standard Bellman error objective with a novel Q-value regularizer. This regularizer works by finding and minimizing (pushing down) erroneously high Q-values for out-of-distribution actions, while a more advanced version also compensates by pushing up Q-values for actions present in the dataset. This makes the learned Q-function inherently "pessimistic".

CQL is theoretically guaranteed to learn a lower bound on the true Q-function, effectively preventing the problematic overestimation. This direct approach leads to state-of-the-art performance across a wide range of tasks on standard benchmarks like D4RL[16] and Atari[8]. It significantly outperforms many previous offline RL methods, demonstrating the ability to ”stitch” together parts of different trajectories to find more optimal paths, and achieving up to five times better results on challenging dexterous manipulation tasks. While conservative, the full CQL method only slightly underestimates Q-values, making it a robust and reliable algorithm.

The effectiveness of CQL depends on the appropriate selection of the regularization weight α. Simpler variants of the algorithm without the compensatory term for in-distribution actions can lead to excessive underestimation. Despite its strong performance, ongoing research continues to explore further improvements.

In summary, policy constraint methods and implicit policy constraint methods both try to constrain the learned policy to remain close to the behavior policy. However, they do not aim to learn a pessimistic Q-function or ensure a lower bound on Q-values. They simply avoid the problematic evaluation of out-of-distribution actions. On the other hand, MOPO and CQL are methods that embrace a conservative and “pessimistic” strategy in order to mitigate the risks associated with going out of distributions and value overestimation by either directly pushing down the Q-values or penalized rewards based on uncertainty. This ensures the learned policies are robust, accepting the trade-off that may lead to sub-optimal but reliable performance.

2.2 Diffusion Models

The following two sections provide the necessary background theory on diffusion and score-based models preceding flow matching. Section 2.2 introduces the formulation of diffusion models, one derived from a variational inference perspective. Section 2.3 explores a more general score-based approach defined by continuous-time stochastic differential equations. Finally, section 2.4 introduces flow matching, a simulation-free method for training Continuous Normalizing Flows based on deterministic Ordinary Differential Equations (ODEs).

2.2.1 Diffusion’s Roots in Variational Bayesian Methods

In probabilistic generative modeling, the common practice is to introduce latent variables z and a parameterized generative model $p_\theta(x)$ with prior $p(z)$ to explain observations x. By Bayes’ rule, the model’s posterior distribution is expressed as:

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}. \tag{2.1}$$

Where the denominator is the marginal likelihood or evidence, calculated by 2.2:

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz. \tag{2.2}$$

The posterior $p(z \mid x)$ explains how likely different latent configurations are given the data, but computing it directly is intractable because calculating the evidence requires integrating over latent space, which explodes in complexity. This intractability means we cannot directly evaluate or differentiate $p(x)$.

Hence, the Bayesian objective is maximizing the marginal likelihood, or evidence of the observed data $\log p_\theta(x)$. It quantifies how well a generative model explains the data overall, marginalizing out the latent space. Variational Bayesian methods provide a practical workaround by introducing a tractable approximation $q_\phi(z \mid x)$, referred to as a recognition model or probabilistic encoder, to the true posterior and maximizing the evidence lower bound (ELBO)[17]:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x, z) - \log q_\phi(z \mid x)\Big], \tag{2.3}$$

or more explicitly shown as:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\phi(z \mid x)\Big]. \tag{2.4}$$

2.2.2 Deep Unsupervised Learning using Nonequilibrium Thermodynamics

Building on principles from non-equilibrium thermodynamics, diffusion probabilistic models systematically and slowly destroy data structure by applying an iterative forward diffusion process that gradually perturbs data $x_0 \sim p_{\text{data}}(x)$ into a tractable Gaussian noise via a Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}). \tag{2.5}$$

This interestingly mirrors physical diffusion processes such as ink dispersing in water or smoke spreading in air. This forward process spreads probability mass across the data landscape, creating training signals far from the original data and grounding the method in the same mathematics as Brownian motion.

A reverse diffusion process is then learned to restore this structure. The ideal reverse process $q(x_{t-1} \mid x_t)$, as mentioned in 2.2.1, is intractable as it depends on the entire data distribution. Dickstein et al. therefore parameterized an approximation p_θ designed to match the tractable posterior $q(x_{t-1} \mid x_t, x_0)$ [18]:

$$p_\theta(x_{t-1} \mid x_t) \;\approx\; q(x_{t-1} \mid x_t, x_0). \tag{2.6}$$

The trained model serves as a local compass that estimates the gradient of the log probability landscape $\nabla_x \log p_t(x)$, which points towards regions of higher data likelihood. Here, $p_t(x)$ denotes the time-varying marginal distribution of data at diffusion step t, obtained by evolving clean samples $x_0 \sim p_{data}$ through the forward noising process. This model does not learn $p_t(x)$ directly, but instead estimate its score function. This connects diffusion to score matching later covered extensively in Section 2.3.

Adapting variational inference, one can derive a variational lower bound on this log-likelihood:

$$\log p_\theta(x_0) \;\geq\; \mathbb{E}_{q(x_{0:T})}\Big[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)\Big]. \tag{2.7}$$

This entire framework can therefore be understood intuitively through the lens of variational inference:

- The fixed forward noising process $q(x_{1:T} \mid x_0)$ serves as the probabilistic encoder, mapping the data x_0 into a latent space of noisy variables $x_{1:T}$.
- The learned reverse process $p_\theta(x_{t-1} \mid x_t)$ acts as the probabilistic decoder, which is trained to reverse the noising process.

This means that training diffusion models amounts to optimizing a diffusion-specific ELBO, which remarkably simplifies to training the decoder to predict the noise added at each step, as demonstrated in the subsequent section.

2.2.3 Denoising Diffusion Probabilistic Models

While earlier works established the connection between diffusion and variational inference, the objective was still complex. Ho et al.'s significantly popularized the framework by reparameterizing the model's learning target [3]. The reverse process learns the transition

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1};\, \mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)), \tag{2.8}$$

which is modeled as a Gaussian with a mean $\mu_\theta(x_t, t)$ and variance $\Sigma_\theta(x_t, t)$. The DDPM authors made two key simplifications: First, they found that fixing the variance to a known constant was sufficient for high-quality results. Second, and most importantly, they reparameterized the mean. Instead of training the network to directly predict the mean of the reverse step, they trained it to predict the noise, ϵ, that was originally added to the clean data. This transforms the complex variational objective into a simple mean-squared error loss between the true noise and the predicted noise:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,\epsilon}\left[\, \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \,\right], \tag{2.9}$$

where ϵ is the true Gaussian noise sampled at a random timestep t, and $\epsilon_\theta(x_t, t)$ is the network's prediction given the noisy image x_t and timestep t. This simplified objective was not only more stable to train but also produced significantly higher-quality samples, which was the major breakthrough that led to the widespread adoption of diffusion models.

2.3 Score-based Models

An alternative and powerful approach to generative modeling is to learn the score function of the data distribution, which is formally defined as the gradient of its log probability $\nabla_x \log p_{\text{data}}(x)$ [19]. This score function creates a vector field that points towards regions of higher data density, providing a trajectory back to the data manifold from any point in the space. Once this score field is learned, new samples can be generated by starting from random noise and iteratively moving in the direction of the score, a process known as Langevin dynamics.

2.3.1 Denoising Score Matching

Estimating the score function directly is difficult. Vincent et al. provided a workaround in Denoising Score Matching (DSM) [19]. The key insight was that the objective of training a simple network to denoise data corrupted with Gaussian noise is equivalent to learning the score of that noise-perturbed data distribution. Given a clean sample x, corruption is defined as

$$\tilde{x} = x + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{2.10}$$

$\tilde{x}$ being a noisy sample. When the corruption is additive isotropic Gaussian noise, the target score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)$ for a noisy sample $\tilde{x}$ given a clean sample x simplifies to $\frac{1}{\sigma^2}(x - \tilde{x})$.

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = \frac{1}{\sigma^2}(x - \tilde{x}) = -\frac{\epsilon}{\sigma}. \tag{2.11}$$

This shows that the target score is directly proportional to the negative noise. This is why instead of a complex score matching objective, one could simply train a denoising network $D_\theta(\tilde{x})$ with a straightforward mean-squared error loss like the one in DDPM:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{x,\tilde{x}}\Big[\, \|D_\theta(\tilde{x}) - x\|^2 \,\Big]. \tag{2.12}$$

This made score estimation practical and stable.

2.3.2 Generative Modeling by Estimating Gradients of the Data Distribution

While DSM worked for a single noise level, it struggled with real-world data that often resides on a low-dimensional manifold, leaving the score undefined in most of the space. Song et al. solved this problem similar to diffusion by perturbing the data with multiple levels of Gaussian noise [20]. They then trained a single Noise Conditional Score Network (NCSN), denoted $s_\theta(x_t, {}_t)$, to jointly estimate the scores for all the different noise-perturbed distributions. This effectively creates a well-defined vector field at all locations and scales.

For generation, annealed Langevin dynamics is introduced. The process starts with pure Gaussian noise and follows the gradient provided by the learned score network s_θ. The noise level σ is slowly annealed from high to low, guiding the sample from the simple noise distribution back to the complex data distribution via an iterative update step:

$$x_{t-1} = x_t + \alpha s_\theta(x_t, \sigma_t) + \sqrt{2\alpha} z_t \tag{2.13}$$

where α is the step size and $z_t \sim \mathcal{N}(0, I)$ is a standard Gaussian noise term.

2.3.3 Score-Based Generative Modeling Through Stochastic Differential Equations

The SDE framework unifies score matching and DDPMs by describing them as discretizations of a single continuous-time process [21].

The forward SDE is defined to transform a complex data distribution into a simple noise prior over a time interval $t \in [0, T]$:

$$dx = \underbrace{f(x,t)}_{\text{drift}}\, dt + \underbrace{g(x,t)}_{\text{diffusion}}\, dW, \tag{2.14}$$

where W is a standard Wiener process and $f(x,t)$ and $g(x,t)$ are the drift and diffusion coefficients, respectively. This describes a continuum of noisy intermediate distributions $\{p_t(x)\}$. Additionally, a corresponding reverse-time SDE that can reverse this noising process exists:

$$dx = \underbrace{\left[f(x,t) - g(x,t)^2 \nabla_x \log p_t(x)\right]}_{\text{drift}} dt + \underbrace{g(x,t)}_{\text{diffusion}}\, d\bar{W}, \tag{2.15}$$

where $d\bar{W}$ is a standard Wiener process run backward in time. Much like the forward SDE, one can intuitively break this equation down into a deterministic drift term and a stochastic diffusion term. And conveniently, the deterministic dynamics depend only on the score function $\nabla_x \log p_t(x)$. Note that for models like DDPMs, the diffusion term simplifies from the general form $g(x,t)$ to $g(t)$ because the noise schedule is designed to be state-independent, as detailed further in A.1. Since this score is intractable, it is approximated by a neural network $s_\theta(x, t)$, which can then be used by numerical solvers to generate samples [6]. This mathematically elegant framework unlocks some new capabilities:

1. Flexible sampling: Because the process is continuous, practitioners are no longer locked into a fixed number of discrete steps. This allows for a direct trade-off between computation speed and sample quality, as any numerical SDE solver can be used to generate samples.

2. The continuous nature of the SDE makes it highly amenable to controllable generation. The reverse sampling process can be guided to solve inverse problems like image inpainting and colorization, often without needing to retrain the model.

3. Exact Log-Likelihoods via an Equivalent ODE: The SDE framework revealed a corresponding deterministic process, described by a neural ordinary differential equation (ODE) known as the probability flow ODE [22]. This breakthrough not only enables deterministic sampling and exact likelihood computation but also provides the direct theoretical inspiration for Flow Matching, which will be detailed in the next section.

2.4 Flow-based Models

Sections 2.2 and 2.3 laid the foundation of simulation-based models, where generating a sample requires simulating a multi-step denoising process. This section now turns to a powerful successor paradigm: the recently proposed flow-based models. Representing the next generation of generative modeling, this framework learns a direct, invertible transformation that can map noise to data in a single, continuous pass. By reframing the generative process around deterministic Ordinary Differential Equations (ODEs), this approach offers substantial advantages, most notably a dramatic reduction in inference time, a key motivation for the planning framework developed in this dissertation.

2.4.1 Normalizing Flows

The core idea of Normalizing Flows is to learn an invertible function f that warps a simple base distribution like a standard Gaussian $p_z(z)$ to a data distribution $p_X(x)$ directly [22]. Using the change of variables formula, the probability density of $x = f(z)$ can be calculated exactly:

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|. \tag{2.16}$$

The $\frac{\partial f^{-1}(x)}{\partial x}$ is the Jacobian matrix of the inverse transformation, and its determinant measures how the function locally stretches or compresses space. For a general, unrestricted neural network, this determinant is computationally infeasible as data dimension scales up.

To solve this, NFs decompose f into a series of transformations $f_1 \circ \cdots \circ f_k$, each designed to have a Jacobian determinant that is easy and efficient to compute. However, this poses architectural constraints as the network cannot be a dense web of all-to-all connections. Specialized blocks like coupling layers are used in RealNVP[23] and Glow [24]. These layers operate by splitting the input data in half and leaving one half frozen while transforming the other. Though this makes training feasible, the necessary rigidity limits the model's expressivity and flexibility.

2.4.2 Continuous Normalizing Flows

The architectural limitations of discrete NFs motivated Continuous Normalizing Flows (CNFs). Instead of a finite stack of layers, a CNF conceptualizes the transformation as a continuous-time process using Ordinary Differential Equations (ODEs) [25]. The entire process can be visualized as a smooth "flow" that gradually warps a simple initial distribution into a complex target data distribution, as illustrated in the figures 2.1.

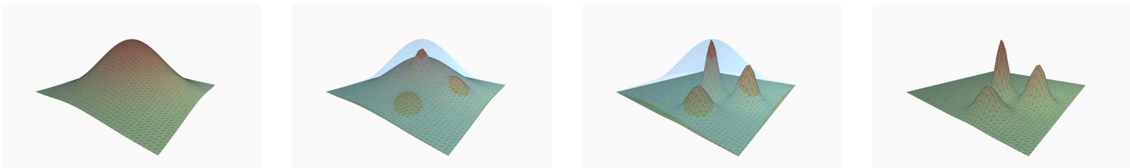

Figure 2.1: An illustration of a Continuous Normalizing Flow. Over time, a simple, unimodal Gaussian distribution (left) is progressively transformed by a learned vector field into a complex, multimodal data distribution (right).

In other words, instead of having the neural network representing the entire transformation, it defines a time-dependent vector field $v(z(t), t, \theta)$. This vector field dictates the direction and speed that any point z moves at any given time t. The dynamics of this transformation can be expressed as:

$$\frac{dz(t)}{dt} = v(z(t), t, \theta). \tag{2.17}$$

This continuous formulation elegantly sidesteps the primary issue of discrete NFs. The change of variables formula from 2.16 now becomes:

$$log(p_X(x)) = log(p_Z(z(0))) - \int_0^1 Tr\left(\frac{\partial v(z(t), t, \theta)}{\partial z(t)}\right) dt. \tag{2.18}$$

Since the trace of a Jacobian is much more efficient to compute than the full determinant, there are no architectural constraints. Any standard neural network architecture like a ResNet [26] can be used to learn the vector field, granting CNF much more flexibility. However, to calculate the likelihood for each training step, a numerical ODE solver must compute the entire trajectory of a data point. Furthermore, computing gradients requires solving a second adjoint ODE. This is computationally and memory intensive, as well as being numerically unstable.

2.4.3 Flow Matching for Generative Models

Flow Matching was introduced to bypass the training bottleneck of CNF by reframing the CNF training objective from complex likelihood computations into a direct regression objective [5].

To construct such an objective, a continuous probability path $p_t(x)$ is first defined. This path smoothly interpolates between a simple prior distribution $p_0(x)$ at $t = 0$ and the data distribution $p_1(x)$ at $t = 1$. The key insight is that any such path has a unique corresponding vector field $u_t(x)$ that generates it. This relationship is governed by the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot [p_t(x)u_t(x)]. \tag{2.19}$$

This equation provides a link between the rate of change of the probability density and the divergence $\nabla$of the probability flux. The divergence measures how much the vector field is "flowing out" of a given point. In essence, the continuity equation formalizes the intuition that if more probability flows away from a region than into it, the density in that region must decrease. This principle then guarantees the existence of a "ground truth" vector field u_t for any defined path p_t. The goal of flow matching is to train a neural network $v(x, t, \theta)$ to learn this underlying vector field, which can be visualized as a time-varying "current" that guides the probability mass, as shown in figure 2.2

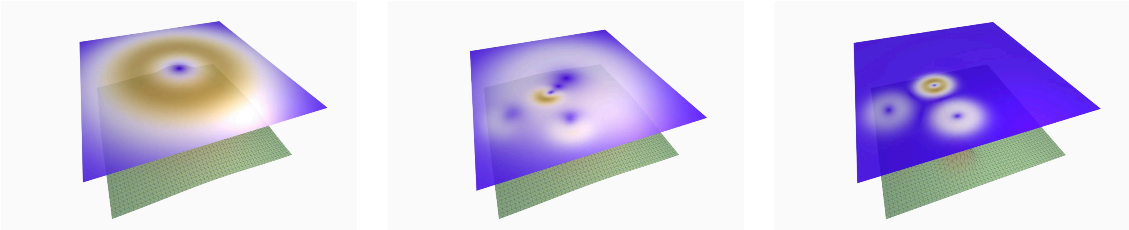

Figure 2.2: A visualization of the learned vector field at different times t. The field represents the direction and magnitude of the "flow" that transports the initial noise distribution (left) towards the target data distribution (right).

The general flow matching objective then becomes a straightforward L2 loss between the model's predicted vector field and the target vector field:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,p_t(x)}\left[\|v(x,t,\theta) - u_t(x)\|^2\right]. \tag{2.20}$$

This objective is simulation free because it only requires sampling a time t and a point x from the known marginal distribution $p_t(x)$ and regressing the model's output onto the target vector [6]. While this general formulation is theoretically convenient, it requires access to both the marginal probability path $p_t(x)$ and the marginal vector field $u_t(x)$. For most paths connecting a simple prior to a complex data distribution, both of these quantities are intractable. This issue is the primary motivation for conditional flow matching, with a practical implementation covered extensively in Section **??** that constructs the path in a specific way to ensure the target vector field is simple and known.

2.5 Planning with Diffusion

Section 2.1 highlighted a core challenge in offline reinforcement learning: calibrating the precise degree of pessimism to prevent value overestimation without creating overly conservative policies. The generative models detailed in Sections 2.2, 2.3 and 2.4 offer a powerful framework for solving this issue.

Among these, diffusion models have been successfully applied to practical trajectory planning problems [2, 4]. Rather than relying on explicit uncertainty penalties, diffusion models learn the underlying behavioral policy itself. These trained models function as a high-fidelity sampler for in-distribution actions, providing a powerful yet implicit constraint on the learned policy. This solves the pessimism calibration problem by ensuring the policy does not stray into unsupported regions of the state-action space. This section now reviews key papers that use this approach for trajectory planning.

2.5.1 Flexible Behavior Synthesis using Diffusion

Model-based methods like MOPO, as referenced in 2.1.3, rely on training a dynamics model to predict future states, but unrolling these learned dynamics models over long horizons often results in compounding prediction errors and leads to out-of-distribution states. This in turn generates adversarial plans that appear optimal but yield poor rewards in practice. Janner et al. addresses these by reconceptualizing planning as a generative modeling problem, training a diffusion model to directly generate entire trajectories [4]. This approach offloads reward optimization and dynamics rollouts into the generative process itself, making planning a sampling problem where guidance can be used to construct desired plans.

The proposed model, Diffuser, operates by a non-autoregressive prediction procedure, generating the entire trajectory simultaneously from Gaussian noise. A trajectory τ is represented as an interleaved sequence of states $s$and actions a of a finite horizon H:

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_H, a_H) \tag{2.21}$$

A fixed forward process $q(\tau^i|\tau^{i-1})$ gradually adds Gaussian noise to the trajectory while a learned denoising process $p_\theta(\tau^{i-1}|\tau^i)$ iteratively refines the noise over N steps to recover trajectory τ^0.

To function as a planner, this sampling process is guided to satisfy task-specific objectives by sampling from a perturbed distribution:

$$\tilde{p}_\theta(\tau) \propto p_\theta(\tau)h(\tau) \tag{2.22}$$

Here, $p_\theta(\tau)$ ensures the plan is realistic and $h(\tau)$ acts as a guidance function that biases it towards the desired outcome. This decoupling of the base model from the goal is implemented in two ways:

1. Guided sampling is used for reward maximization, where the mean of the reverse process is shifted at each step by the gradient of the reward function $\nabla\mathcal{J}(\mu)$.

2. Inpainting is used for hard constraints, such as reaching a specific goal, where known values like the start and end states are fixed in the final denoised trajectory.

The use of generative sampling as planning allows for flexible behavior synthesis by composing the learned model with various guidance functions at test time. As a result, a single trained model can be adapted to diverse tasks without retraining, including:

- Offline reinforcement learning, where the model can be guided by different reward functions to perform competitively with specialized techniques.
- Goal-seeking policies for long-horizon and sparse-reward problems by fixing the start and end observations via inpainting. This was shown to be effective in complex maze environments.
- Arbitrary user-defined constraints, such as completing complex block-stacking configurations, by combining multiple perturbation functions.

2.5.2 Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

While Planning with diffusion treats planning as a conditional generation problem over entire state–action trajectories, Diffusion Policy focus on learning a reactive, visuomotor policy via imitation learning [2]. The paper addresses the core challenges of behavior cloning, where standard explicit policies (e.g., direct regression) struggle to model complex, multimodal action distributions, and implicit policies (e.g., energy-based models) suffer from training instability due to reliance on negative sampling. Chi et al. proposes to overcome these issues by representing the policy itself as a conditional denoising diffusion process.

This formulation can be understood as a practical, discretized implementation of the general reverse-time SDE introduced in equation 2.15. Instead of directly mapping an observation O_t to an action sequence A_t, the neural network $\epsilon_\theta(O_t, A_t^k, k)$ learns to predict the noise present in a noised action sequence A_t^k at diffusion step k.

At inference time, the policy starts with an action sequence A_t^K sampled from pure Gaussian noise and iteratively refines it over K steps to produce a clean, executable action sequence A_t^0. The denoising update at each step is given by:

$$A_t^{k-1} = \alpha \left(A_t^k - \gamma \epsilon_\theta(O_t, A_t^k, k) \right) + \mathcal{N}(0, \sigma^2 I). \tag{2.23}$$

The network is optimized with a simple mean-squared error loss between the true and predicted noise:

$$\mathcal{L} = \mathrm{MSE}(\epsilon_k, \epsilon_\theta(O_t, A_t^0 + \epsilon_k, k)). \tag{2.24}$$

This formulation learns the score function of the conditional action distribution $p(A_t \mid O_t)$, which provides excellent distribution expressivity while remaining stable to train, unlike prior implicit methods.

To make this formulation practical for real-world robotics, Chi et al. combines the prediction of action sequences with receding horizon control, where the policy predicts a future sequence of actions (T_p) but only executes a shorter subsequence (T_α) before replanning. Second, it uses an efficient visual conditioning scheme where visual features are extracted once per inference cycle for real-time performance. Finally, the paper proposes Time-series diffusion transformer architecture to better handle tasks requiring high-frequency action changes. These contributions allow Diffusion Policy to consistently outperform prior state-of-the-art methods across numerous robot manipulation benchmarks.

2.6 Other Relevant Works

2.6.1 Q-Score Matching (QSM)

Q-Score Matching offers a novel training objective that reframes policy optimization from a geometric perspective[27]. The core insight is to directly align the score function of the diffusion policy, $\nabla \log \pi_\theta(a|s)$, with the action-gradient of the learned Q-function, $\nabla_a Q(s, a)$. In other words, the policy is trained such that its vector field of log-probabilities is iteratively "pushed" to match the direction of steepest ascent in the Q-value landscape.

This approach provides a direct, analytical objective for policy improvement that completely avoids the need to differentiate through the diffusion sampling chain. By exploiting the inherent score-based structure of the diffusion model, QSM enables the stable and sample-efficient learning of complex, multi-modal policies that can more effectively cover the modes of the optimal action distribution.

2.6.2 Diffusion Steering via Reinforcement Learning (DSRL)

In contrast to modifying the training objective, Diffusion Steering via Reinforcement Learning (DSRL) is an inference-time optimization technique [28]. It leaves the pre-trained diffusion policy fixed and instead focuses on the initial latent noise, $wN(0, I)$, which is the seed for the denoising process. The key idea is to train a secondary, lightweight "steering" policy that learns to select an initial noise vector w that, when denoised by the fixed diffusion model, produces a high-value action.

This method also tries to address counterfactual query: "What if a different initial noise had been chosen?" By learning to navigate the latent noise space, DSRL can rapidly steer the powerful generative prior towards actions that maximize rewards for a specific task. This allows for extremely fast and sample-efficient policy adaptation without the computational cost or potential instability of fine-tuning the large, pre-trained diffusion model itself.

2.6.3 Flow Q-Learning

A third, highly effective approach to the policy extraction problem is presented in Flow Q-Learning (FQL) [29]. This method elegantly circumvents the challenges of direct policy optimization by decoupling the learning of the expressive behavioral prior from the value maximization task itself. FQL uses a "teacher-student" distillation framework [30].

First, an iterative, flow-matching policy is trained solely with behavioral cloning (BC) to serve as an expressive "teacher" model that perfectly captures the complex, multimodal action distributions in the expert dataset. The reinforcement learning objective is then offloaded to a separate, simple one-step "student" policy. This student policy is trained to maximize the learned Q-function, but it is simultaneously regularized by a distillation loss that forces its output to stay close to that of the powerful flow-matching teacher. This strategy is extremely clever because it completely avoids the unstable and computationally expensive backpropagation through the iterative ODE solver, and the final deployed policy is the efficient one-step student. Costly iterative sampling at inference time is therefore completely avoided. FQL thus achieves the best of both worlds, a distributional expressivity of a flow model with the training stability and inference speed of a simple policy.

3 METHODOLOGY

This chapter details the methodology for the proposed Flow Planner framework, a novel hierarchical framework for long-horizon trajectory planning. This chapter is structured to build from foundational theory to the specific contributions of this work.

Section 3.1 first introduces the theoretical underpinnings of Conditional Flow Matching (CFM), a powerful, recently proposed paradigm for generative modeling. Building upon this, Section 3.2 introduces a clever reformulation of the CFM objective, specializing this framework to the complex domains of robotic trajectory planning. This section goes over the general formulations for each planner component developed in this dissertation. Finally, Section 3.3 describes how these components are integrated into the complete hierarchical framework.

The formulations presented herein are adapted from the convention from ”An Introduction to Flow Matching and Diffusion Models” by MIT CSAIL [6], as well as the “Flow Matching Guide and Code” provided by Meta [7].

3.1 Conditional Flow Matching Formulation

Recall in Section 2.4.3, the general Flow Matching objective 2.20 is intractable because it requires access to the marginal probability path $p_t(x)$ and its corresponding vector field $u_t(x)$. Conditional Flow Matching (CFM) is a practical implementation that solves this problem by defining the probability path such that it yields a tractable target vector field. This transforms the objective into a stable and efficient regression problem.

3.1.1 CFM Training

CFM constructs the path as a simple linear interpolation between a point sampled from a prior distribution $x_0 \sim p_0$ and a point from the data distribution $x_1 \sim p_1$. The prior is usually standard Gaussian noise $p_0(x) = \mathcal{N}(0, I)$. The point on the path at any time $t \in [0, 1]$ is defined as:

$$x_t = (1 - t)x_0 + tx_1. \tag{3.1}$$

This specific construction is a type of Gaussian probability path referred to as the CondOT (Conditional Optimal Transport) path [7]. The ground truth vector field requiring to move a particle along this straight path is constant at all times, and it is simply the vector connecting the two endpoints:

$$u_t(x_t) = x_1 - x_0. \tag{3.2}$$

This replaces the intractable integral of the general formulation with a simple difference of two vectors, which are readily available during training. By substituting this simplified target vector field into the general objective in 2.20, one arrives at the final CFM loss function. The goal is to train a neural network $v_\theta(x_t, t, c)$ to predict the target vector $(x_1 - x_0)$ given the interpolated point x_t and time t, and any optional conditioning variables c. This objective is a straightforward L2 regression loss:

$$L_{CFM}(\theta) = E_{t,p_0(x_0),p_1(x_1)}[\|v_\theta((1-t)x_0 + tx_1, t, c) - (x_1 - x_0)\|^2]. \tag{3.3}$$

A single training step is summarized in Algorithm 1

Algorithm 1 CFM Training

Require: Dataset of samples $\mathbf{x}_1 \sim p_{\text{data}}$, neural network v_θ, prior distribution p_0 (e.g. $\mathcal{N}(0, I)$)

```
for each training step do
    x_1 ~ p_data                          ▷ Sample a data point
    x_0 ~ p_0                             ▷ Sample a noise vector
    t ~ Unif[0, 1]                        ▷ Sample a random time
    x_t ← (1 − t)x_0 + t x_1              ▷ Compute interpolated point
    u ← x_1 − x_0                         ▷ Compute target vector
    L(θ) ← ‖v_θ(x_t, t, c) − u‖^2         ▷ Compute the loss
    θ ← θ − η∇_θ L(θ)                     ▷ Gradient descent update
end for
```

This is a simulation-free objective, hence it only requires sampling one data point, one noise vector, and a time to perform a standard regression update. It avoids the costly ODE solves of traditional CNF training and forms the foundational recipe for the specific planners detailed in the following sections. While the CondOT path is a common choice, the implementation in this dissertation utilizes a more general affine probability path. This formulation, known as the ”Affine Conditional Flow Matching” variant in Albergo et al. [cite here], defines an exact conditional distribution over the interpolants instead of a single deterministic interpolation. This preserves key probabilistic properties, leading to more stable training dynamics and a closer theoretical alignment with Schrödinger Bridge formulations [31]. The detailed derivation of the base CFM objective is provided in Appendix A.1.

3.1.2 CFM Inference

While the training process for CFM is simulation-free, inference requires simulating the learned dynamics. The goal of inference is to generate a new sample x_1 that approximates a draw from the true distribution by solving an Ordinary Differential Equation (ODE). The trained neural network $v_\theta(x_t, t, c)$ now serves as an approximation of the true (but intractable) marginal vector field that transforms noise into data. Generation begins by sampling an initial point from the prior distribution $\mathbf{x}_0 \sim p_0(\mathbf{x})$. This point is then evolved from $t = 0$ to $t = 1$ by solving the following ODE:

$$\frac{d\mathbf{x}t}{dt} = v_\theta(\mathbf{x}_t, t, c). \tag{3.4}$$

The solution to this ODE, x_1, is the final generated sample. Since this ODE rarely has an analytical solution, it must be solved numerically. A common choice is the simple **Euler method**, detailed in Algorithm 2.

Algorithm 2 Euler Method

Require: Neural network v_θ, prior distribution p_0, condition c, number of steps N

```
x_0 ~ p_0(x)                                  ▷ Sample an initial point
Δt ← 1/N
for i = 0 to N − 1 do
    t ← i · Δt
    x_{t+Δt} ← x_t + v_θ(x_t, t, c) · Δt        ▷ Take a step along vector field
end for
return x_1
```

It is worth noting that for Stochastic Differential Equations (SDEs), the corresponding simple solver is the Euler-Maruyama method. For the experiments in this dissertation, a more advanced solver, **the midpoint method** (a 2nd-order Runge-Kutta scheme) is employed [7]. This choice offers a better trade-off between computational cost and numerical stability, leading to higher-quality samples compared to the basic Euler method.

3.2 Planners Formulation

The general Conditional Flow Matching (CFM) objective from Section 3.1 provides a simulation-free training recipe. While this framework has been successfully specialized for a range of generative modeling tasks, particularly in domains such as image and video synthesis, its application to trajectory planning remains largely unexplored.

This section details the novel specialization of the CFM framework for trajectory planning. To present the methodology in a unified manner, this section first presents a generalized planner formulation and then details how this single framework is instantiated for three different planners: Joint Planner (JP), Waypoint Planner (WP), and Action Planner (AP). Note the distinction between the overall task horizon T, which can be arbitrarily long, and the fixed planning horizon of the model H, which is a number of steps the generative model is trained to predict. This distinction is key to how the framework manages long-horizon problems.

3.2.1 Generalized Planner Formulation

Let τ represent a generic trajectory data sample. This sample can be a state-action sequence, a state-only sequence, or an action-only sequence over the planning horizon H and let c be a set of conditioning variables. The goal is to learn a conditional distribution $p(\tau \mid c)$ by training a neural network $v_\theta(\tau_t, t, c)$. The generalized CFM objective for any planner is given by:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\tau,\tau_{noise}} \| v_\theta(\tau_t, t, c) - (\tau - \tau_{noise}) \|, \tag{3.5}$$

where τ is a ground-truth trajectory from the dataset, $\tau_{noise} \sim \mathcal{N}(0, I)$, and

$$\tau_t = (1-t)\tau_{\text{noise}} + t\tau. \tag{3.6}$$

3.2.2 Planner Specializations

The three planners implemented below are specific instances of the above generalized formulation, differing only in their choice of trajectory data τ and conditioning variables c.

3.2.2.1 Joint State-Action Planner (JP)

The Joint Planner is designed to generate a complete, long-horizon strategy that includes both states and actions. It learns the conditional distribution over entire state-action trajectories $p(\tau \mid c)$, where $c = (s_{start}, s_{goal})$. A JP outputs a trajectory designed over the planning horizon H:

$$\tau = (s_0, a_0, ..., s_H, a_H). \tag{3.7}$$

This formulation is inspired by the inpainting-based planning approach of Janner et al, who demonstrated the feasibility of generating trajectories within a diffusion model framework [4]. However, whereas their method relied on a separate guidance function $\mathcal{J}_\phi$ to steer the sampling process, my CFM-based implementation models the conditional distribution directly. While this end-to-end approach is theoretically powerful (and intuitive) for open-loop planning, modeling the full joint state-action space over a long horizon is often practically infeasible. The subsequent planners are designed to mitigate this by decoupling the problem.

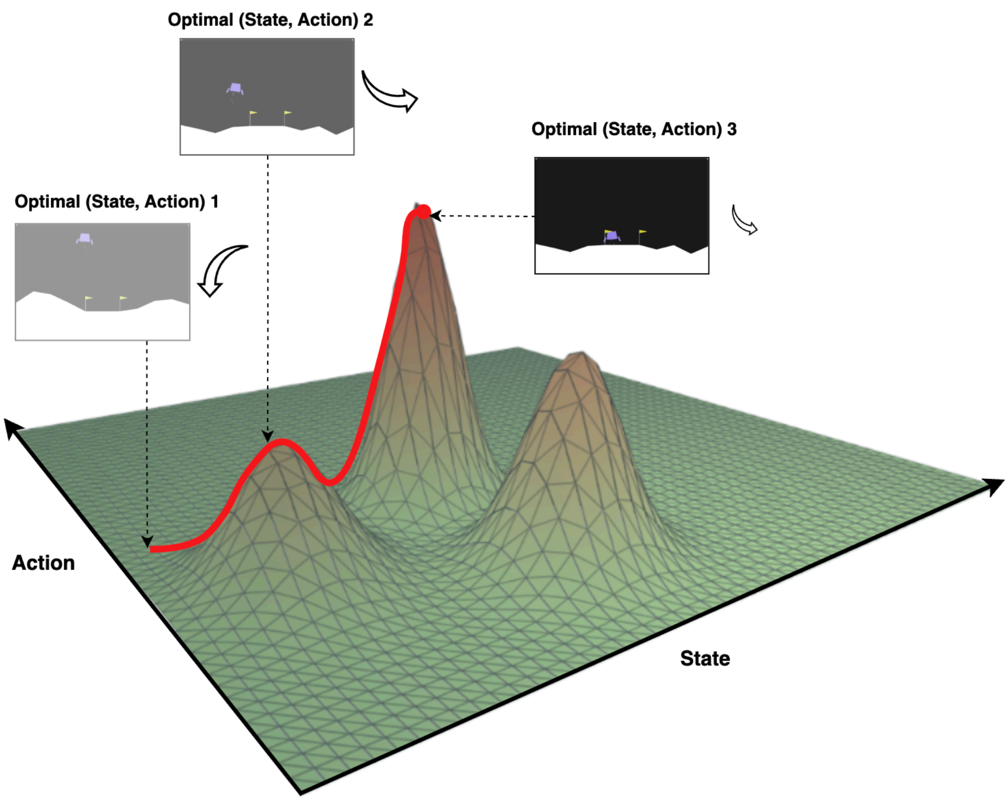

Figure 3.1: Joint State-Action Planner Visualized

3.2.2.2 State-Only Waypoint Planner (WP)

I proceed to decouple the high-level pathfinding from the low-level control. The Waypoint Planner generates a long-horizon strategic plan consisting only of states, which serve as a sequence of waypoints for a low-level controller, also defined over the planning horizon H:

$$\tau_s = (s_0, ..s_H). \tag{3.8}$$

The Waypoint Planner learns the conditional distribution over state-only trajectories, $p(\tau_s \mid c)$, where the conditioning $c = (s_{start}, s_{goal})$ remains the same.

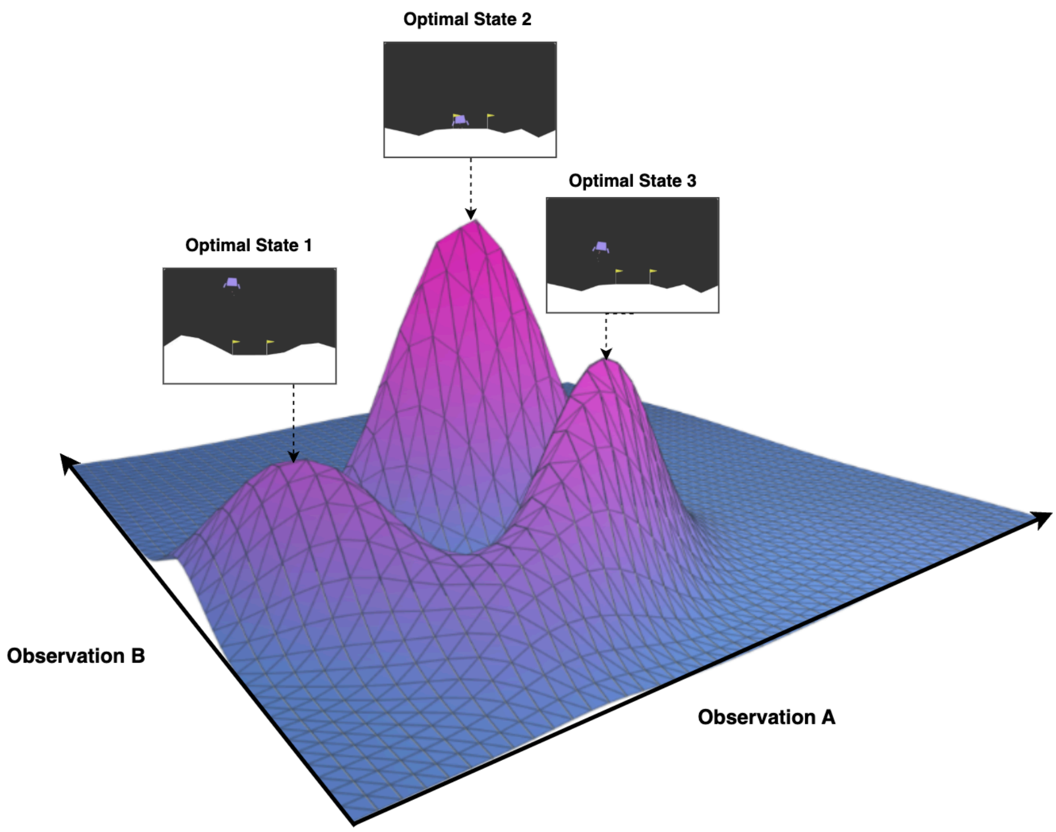

Figure 3.2: State-Only Waypoint Planner Visualized

3.2.2.3 Action Planner (AP)

Inspired by the work of Chi et al.'s on Diffusion policy, the Action Planner is designed to be a reactive, low-level policy [2]. Its role is to generate a short-horizon sequence of actions that guides the agent from its current state to a nearby waypoint, typically provided by a high-level planner like the WP.

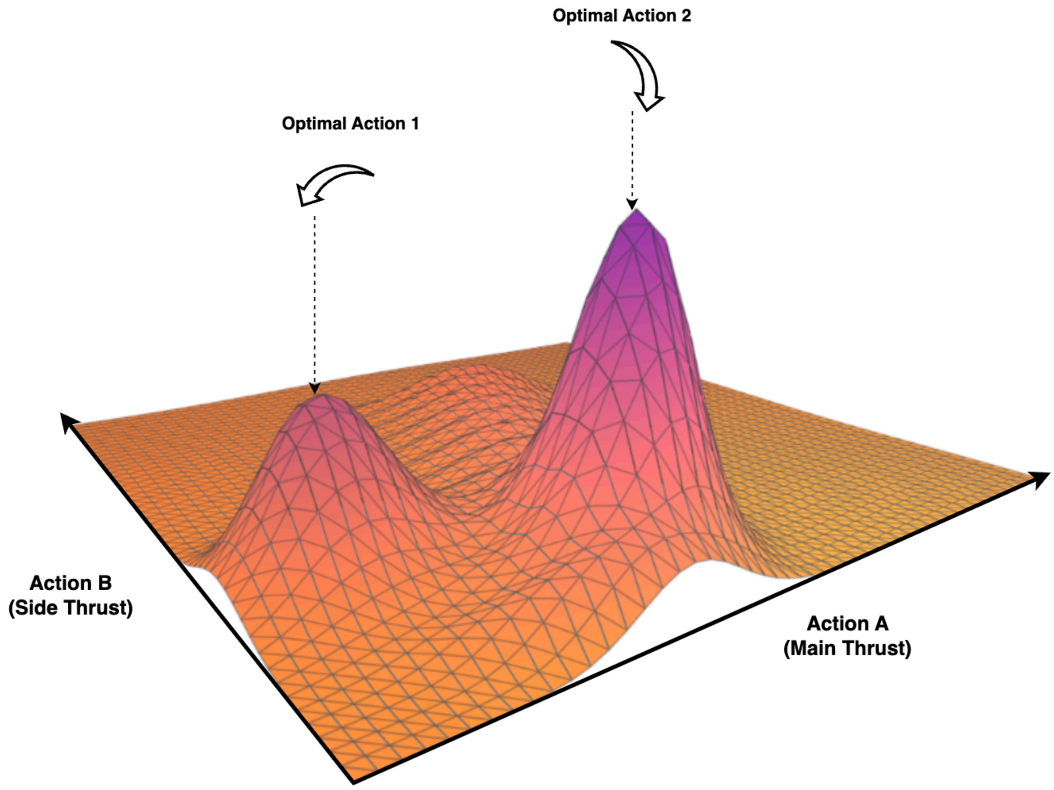

Figure 3.3: Action Planner Visualized

It learns the conditional action distribution $p(\tau_a \mid c)$, where the conditioning is $c = (s_{current}, s_{waypoint})$. Note the action sequence is defined over a short action horizon H_a , where $H_a \ll H$.

$$\tau_a = (a_0, \dots, a_{H_a}). \tag{3.9}$$

A summary of each planner can be found in Table 3.1

Table 3.1: Overview of planners, their models, trajectory data, and conditioning variables.

Planner	**Model**	**Trajectory Data (τ)**	**Conditioning Variables (c)**
Joint Planner (JP)	v_θ^{JP}	$(s_0, a_0, \ldots, s_H, a_H)$	$(s_{\text{start}}, s_{\text{goal}})$
Waypoint Planner (WP)	v_θ^{WP}	$(s_0, \ldots, s_H)$	$(s_{\text{start}}, s_{\text{goal}})$
Action Planner (AP)	v_θ^{AP}	$(a_0, \ldots, a_{T_a})$	$(s_{\text{current}}, s_{\text{waypoint}})$

3.3 Flow Planner Framework

The individual planners in Section 3.2 are composed into a hierarchical framework that tackles long-horizon tasks by decoupling high-level strategic reasoning from low-level reactive control. This section details the framework's operational flow and motivates the key design choices. The architecture pairs a state-only Waypoint Planner (WP) at the high level with an Action Planner (AP) at the low level, executed within a Model Predictive Control (MPC) loop. MPC follows a receding-horizon scheme: at each step a finite-horizon open-loop plan is computed; only its first action is applied, after which the horizon is shifted and the optimization re-solved [32].

3.3.1 Hierarchical Planning Process

The planning process unfolds in two distinct stages: an initial strategic planning phase and a continuous, reactive control loop.

1. **Global Plan Generation**: At the beginning of a planning cycle, the Waypoint Planner (WP) is invoked to generate a strategic plan, $\tau_s = (s_0, \ldots, s_H)$, over its fixed planning horizon H. The conditioning signal for this planner, $c = (s_{start}, s_{goal})$ can be formulated in two ways, each with distinct purpose:

   - **Final Goal Conditioning**: The planner can be conditioned on the final goal of the entire task horizon, where $s_{goal} = s_T$. This provides the model with the ultimate objective, encouraging it to generate a plan that is globally directed towards the final destination.

   - **Waypoint Goal Conditioning**: Alternatively, the planner can be conditioned on an intermediate goal at the end of its planning window, $s_{goal} = s_H$. In this receding-horizon approach, the planner solves a shorter, more immediate problem. For tasks whose total duration exceeds the planning horizon ($T \gg H$), the WP can be invoked in a low-frequency receding-horizon manner as well to generate the next segment of the plan once the current one is near completion.

2. **Reactive MPC Loop**: The system then enters a high-frequency MPC loop to execute the plan:

   (a) The agent's current state, $s_{current}$, is observed.

   (b) The next immediate waypoint, $s_{waypoint}$, is selected from the global plan τ_s.

   (c) The AP is conditioned on both the current state and the target waypoint, $c = (s_{current}, s_{waypoint})$ and generates a short-horizon action sequence, $\tau_a = (a_0, \ldots, a_{T_a})$.

   (d) The first action a_0 from this sequence is executed in the environment.

   (e) The process repeats, with the agent progressing through the waypoints of the global plan.

The full procedure is detailed in Algorithm 3.

Algorithm 3 Hierarchical Planning

Require: Trained Waypoint Planner v_θ^{WP}, Trained Action Planner v_θ^{AP}, initial state $\mathbf{s}_{\text{start}}$, goal state $\mathbf{s}_{\text{goal}}$

Ensure: Action sequence leading to $\mathbf{s}_{\text{goal}}$

$\tau_s \leftarrow \text{sample}\Big(v_\theta^{WP} \;\mid\; \mathbf{s}_{\text{start}}, \mathbf{s}_{\text{goal}}\Big)$ ▷ Generate high-level plan
$i \leftarrow 1$
$\mathbf{s}_{\text{current}} \leftarrow \mathbf{s}_{\text{start}}$
while not terminated **do**
  $\mathbf{s}_{\text{waypoint}} \leftarrow \tau_s[i]$
  $\mathbf{A}_{\text{seq}} \leftarrow \text{sample}\Big(v_\theta^{AP} \;\mid\; \mathbf{s}_{\text{current}}, \mathbf{s}_{\text{waypoint}}\Big)$
  $a_0 \leftarrow \mathbf{A}_{\text{seq}}[0]$
  $\mathbf{s}_{\text{next}} \leftarrow \text{Execute}(a_0)$ ▷ Execute action and observe next state
  $\mathbf{s}_{\text{current}} \leftarrow \mathbf{s}_{\text{next}}$
  if distance$(\mathbf{s}_{\text{current}}, \mathbf{s}_{\text{waypoint}}) <$ threshold **then** ▷ Check if waypoint was reached
    $i \leftarrow i + 1$
  end if
end while

3.3.2 Framework Design Philosophy

The design of the Flow Planner framework is motivated by the inherent limitations of reactive policies. While powerful for short-horizon tasks, the Diffusion Policy framework proposed by Chi et al can exhibit a "gold fish effect" where a limited observation history makes it difficult to maintain coherence over long durations [4]. This is particularly challenging in tasks with long-horizon multimodality, where the policy may lack sufficient history to disambiguate between different strategic modes, leading to indecisive or "stuck" behaviors.

On the other hand, end-to-end approaches that excel in open-loop planning like the inpainting-based planner proposed by Janner et al. runs into a different scalability issue [2]. While theoretically capable of generating long plans, modeling the entire joint state-action space over a long horizon is often practically infeasible due to the sample complexity and computational cost associated with such high-dimensional model inputs.

Therefore, the Flow Planner Framework is centered around a "hierarchical decomposition" philosophy designed to address both of these challenges simultaneously. It is also worth noting that it is grounded in probabilistic factorization. Using the chain rule of probability, one can factorize the original complex joint state-action space distribution into two simpler conditional parts:

$$p(\tau_s, \tau_a \mid c) = \underbrace{p(\tau_s \mid c)}_{\text{Waypoint Planner}} \cdot \underbrace{p(\tau_a \mid \tau_s, c)}_{\text{Action Planner}} . \tag{3.10}$$

4 DATASET PREPARATION

You need to work out how many technical chapters you will require, possibly more than two in this template. Normally your final chapter before the conclusion will include some clear results or proof of concepts from the end goals of your dissertation and it is important to document these well and show the value of what you have achieved. Frequently many dissertations fail to document this information correctly because it is rushed at the end so do ensure you allow time to write it properly.

4.1 Section

Text to insert

4.2 Section

Text to insert – but ensure the last section really does show the value of your work and where you have got to.

5 EXPERIMENTAL RESULTS

5.1 Architectural Selection

5.2 Modeling Joint Distribution

5.3 Testing Different Conditioning Signals

5.4 Hierarchical Planning

5.4.1 Global Planner

5.4.2 Local Planner

6 CONCLUSIONS

Your conclusion should summarise what you have carried out in this work to date, what you have achieved and what you are on course to achieve. Also it should explain what was documented in this dissertation. Note that someone may read this chapter without bothering to read the earlier chapters so there is no problem with repeating what you may have already written earlier. Indeed no new information should be in this part of the conclusion. It should reference back to what is already there.

6.1 Evaluation

Stand back and evaluate what you have achieved and how well you have met the objectives. Evaluate your achievements against your objectives in section 1.2. Demonstrate that you have tackled the project in a professional manner.

6.2 Future Work

Discuss what further work could be carried out from this work and relate it to what has been carried out.

BIBLIOGRAPHY

[1] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems," Nov. 2020.

[2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," Jun. 2023.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," Dec. 2020.

[4] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with Diffusion for Flexible Behavior Synthesis," Dec. 2022.

[5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," Feb. 2023.

[6] P. Holderrieth and E. Erives, "An Introduction to Flow Matching and Diffusion Models," Jun. 2025.

[7] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, "Flow Matching Guide and Code," Dec. 2024.

[8] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[9] O. G. Younis, R. Perez-Vicente, J. U. Balis, W. Dudley, A. Davey, and J. K. Terry, "Minari," Sep. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.13767625

[10] A. Kumar, J. Fu, G. Tucker, and S. Levine, "Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction," Nov. 2019.

[11] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," Nov. 2019.

[12] A. Nair, A. Gupta, M. Dalal, and S. Levine, "AWAC: Accelerating Online Reinforcement Learning with Offline Datasets," Apr. 2021.

[13] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, and T. Ma, “MOPO: Model-based Offline Policy Optimization,” Nov. 2020.

[14] M. Janner, J. Fu, M. Zhang, and S. Levine, “When to Trust Your Model: Model-Based Policy Optimization,” Nov. 2021.

[15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-Learning for Offline Reinforcement Learning,” Aug. 2020.

[16] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4rl: Datasets for deep data-driven reinforcement learning,” 2020.

[17] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” Dec. 2022.

[18] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” Nov. 2015.

[19] P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, Jul. 2011.

[20] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” Oct. 2020.

[21] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” Feb. 2021.

[22] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, “Normalizing Flows for Probabilistic Modeling and Inference.”

[23] L. Dinh, J. Sohl-Dickstein, and S. Bengio, “Density estimation using Real NVP,” Feb. 2017.

[24] D. P. Kingma and P. Dhariwal, “Glow: Generative Flow with Invertible 1x1 Convolutions,” Jul. 2018.

[25] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural Ordinary Differential Equations,” Dec. 2019.

[26] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Dec. 2015.

[27] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a Diffusion Model Policy from Rewards via Q-Score Matching,” Feb. 2025.

[28] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering Your Diffusion Policy with Latent Space Reinforcement Learning,” Jun. 2025.

[29] S. Park, Q. Li, and S. Levine, “Flow Q-Learning,” May 2025.

[30] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” Mar. 2015.

[31] A. Tong, N. Malkin, K. Fatras, L. Atanackovic, Y. Zhang, G. Huguet, G. Wolf, and Y. Bengio, “Simulation-free Schrödinger bridges via score and flow matching,” Mar. 2024.

[32] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, Jun. 2000.

[33] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “\$$\pi$_0\$: A Vision-Language-Action Flow Model for General Robot Control,” Nov. 2024.

[34] F. Chollet, M. Knoop, G. Kamradt, and B. Landers, “ARC Prize 2024: Technical Report,” Jan. 2025.

[35] W. Fan, A. Y. Zheng, R. A. Yeh, and Z. Liu, “CFG-Zero*: Improved Classifier-Free Guidance for Flow Matching Models,” Apr. 2025.

[36] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit Behavioral Cloning,” Sep. 2021.

[37] Y. Gao, J. Huang, Y. Jiao, and S. Zheng, “Convergence of Continuous Normalizing Flows for Learning Probability Distributions,” Mar. 2024.

[38] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean Flows for One-step Generative Modeling,” May 2025.

[39] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud, "FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models," Oct. 2018.

[40] P. Guo and A. G. Schwing, "Variational Rectified Flow Matching," Feb. 2025.

[41] D. Ha and J. Schmidhuber, "World Models," Mar. 2018.

[42] A. Hyvarinen, "Estimation of Non-Normalized Statistical Models by Score Matching."

[43] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An Open-Source Vision-Language-Action Model," Sep. 2024.

[44] I. Kostrikov, A. Nair, and S. Levine, "Offline Reinforcement Learning with Implicit Q-Learning," Oct. 2021.

[45] X. Liu, C. Gong, and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," Sep. 2022.

[46] N. Ma, S. Tong, H. Jia, H. Hu, Y.-C. Su, M. Zhang, X. Yang, Y. Li, T. Jaakkola, X. Jia, and S. Xie, "Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps," Jan. 2025.

[47] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning," Dec. 2017.

[48] S. Park, K. Frans, B. Eysenbach, and S. Levine, "OGBench: Benchmarking Offline Goal-Conditioned RL," Feb. 2025.

[49] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven Exploration by Self-supervised Prediction," May 2017.

[50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," Aug. 2017.

[51] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction."

[52] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning,” Aug. 2023.

[53] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori, “Hierarchical Reasoning Model,” Aug. 2025.

[54] ——, “Hierarchical Reasoning Model,” Aug. 2025.

[55] Ç. Yıldız, M. Heinonen, and H. Lähdesmäki, “Continuous-Time Model-Based Reinforcement Learning,” Jun. 2021.

[56] M. Zhou, Y. Gu, and Z. Wang, “Few-Step Diffusion via Score identity Distillation,” May 2025.

[57] M. Zhou, H. Zheng, Z. Wang, M. Yin, and H. Huang, “Score identity Distillation: Exponentially Fast Distillation of Pretrained Diffusion Models for One-Step Generation,” May 2024.

A APPENDIX

A.1 Derivation of the Conditional Flow Matching Objective